🎯 CI/CD Pipeline Demo Script

Fabric + GitHub Actions Customer Presentation

Table of Contents

PART 1: Workflow Triggers (Lines 1-38)

Customer Requirement

> **Step 2 — Pull Request:** Developer opens a PR to merge into main, GitHub Actions automatically runs PR Validation

Implementation

```yaml
name: 🚀 Model Training Pipeline - Dev → Test → Prod

on:
  # PR Validation
  pull_request:
    branches: [main]
    paths:
      - 'model_training1.Notebook/**'
      - 'notebooks/model_training.ipynb'
      - 'pipelines/customer_analytics_pipeline.json'
      - 'check_data_quality.py'
```

Demo Script

"This is Step 2 from your requirements - when a developer opens a PR, GitHub Actions automatically runs. Notice the `paths:` filter - we only trigger on relevant changes, not every file modification."

🎱 TRAP QUESTION #1

> **Customer:** "What if someone pushes directly to main without a PR? Won't that bypass validation?"

YOUR ANSWER:

- "Great question! That's why we implement GitHub branch protection rules."
- "You configure main branch to:"
  - Require pull request reviews
  - Require status checks to pass (our PR validation)
  - Prevent force pushes
- "This is separate from the workflow file - it's a GitHub repository setting."
- "Would you like me to show you how to configure that after we review the workflow?"

Auto-Deploy to DEV

```
# Auto-deploy to DEV on merge
push:
  branches: [main]
  paths:
    - 'model_training1.Notebook/**'
    - 'notebooks/model_training.ipynb'
    - 'pipelines/customer_analytics_pipeline.json'
```

Demo Script

"This is Step 3 - the moment a PR merges to main, we automatically deploy to Dev. Notice we use the same `paths:` filter for consistency."

TRAP QUESTION #2

> **Customer:** "What if multiple developers merge at the same time? Will deployments conflict?"

YOUR ANSWER:

- "Excellent concern! GitHub Actions handles concurrency automatically:"
  - By default, multiple workflow runs queue sequentially
  - We can add `concurrency:` groups to cancel in-progress runs if needed
  - For Fabric, we also implement workspace locks in our deployment scripts
- "Would you like me to add a concurrency group to prevent deployment collisions? It's one line of code."

Example:

```
concurrency:
  group: deploy-dev-${{ github.ref }}
  cancel-in-progress: true
```

Manual Deployment for TEST/PROD

```
    # Manual deployment triggers for TEST and PROD
    workflow_dispatch:
      inputs:
        environment:
          description: 'Target Environment'
          required: true
          type: choice
          options:
            - dev
            - test
            - prod
        deployment_reason:
          description: 'Reason for deployment'
          required: true
          type: string
        change_ticket:
          description: 'Change Management Ticket (PROD only)'
          required: false
          type: string
```

Demo Script

"This is Step 4 from your requirements - promotion to Test & Prod. Notice it's `workflow_dispatch` - meaning manual trigger with approval gates. The `type: choice` creates a dropdown, preventing typos like 'tets' or 'production'."

TRAP QUESTION #3

> **Customer:** "How do we enforce that only certain people can deploy to Production?"

YOUR ANSWER: "Perfect question - this is your governance requirement. GitHub provides Environment Protection Rules:"

Configuration Steps:

1. Go to Settings → Environments → Create 'production' environment
2. Add Required Reviewers (specific people or teams)
3. Add Wait Timer (e.g., 5 minutes for incident rollback window)
4. Restrict deployment branches (only main can deploy to prod)

Implementation:

- The `environment: name: production` line in the workflow (line 330) enforces these rules
- The workflow literally pauses and waits for approval before proceeding

---

PART 2: PR Validation Job (Lines 47-109)

Customer Requirement

> **PR Validation:** Linting, JSON schema checks, Notebook integrity checks, Custom rules

Implementation

```
  pr-validation:
    name: 🔍 PR Validation
    if: github.event_name == 'pull_request'
    runs-on: ubuntu-latest

    steps:
      - name: 📥 Checkout code
        uses: actions/checkout@v4

      - name: 🐍 Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: ${{ env.PYTHON_VERSION }}
          cache: 'pip'

      - name: 📦 Install dependencies
        run: |
          pip install -r requirements.txt
          pip install black flake8 nbqa pytest nbformat jupyter
```

Demo Script

"This job ONLY runs on PRs - notice the `if:` condition. It runs on GitHub-hosted runners, so zero infrastructure cost for you."

Linting Requirement

```
      - name: 🎨 Validate Python formatting
        run: |
          black --check scripts/ *.py || echo "⚠️ Format issues found"
          echo "☑️ Code formatting checked"
```

Demo Script

"This is your Linting requirement - using Black, the industry standard Python formatter."

🏐 TRAP QUESTION #4

> **Customer:** "What if we use different linting standards than Black?"

YOUR ANSWER: "Great point! Black is opinionated but configurable. You have options:"

Option 1: Configure Black

```
# pyproject.toml
[tool.black]
line-length = 88
target-version = ['py310']
include = '\.pyi?$'
exclude = '''
/(
    \.git
  | \.venv
  | build
  | dist
)/
'''
```

Option 2: Use Flake8

```
- name: 🎨 Lint with Flake8
  run: |
    flake8 scripts/ --max-line-length=120 --config=.flake8
```

Option 3: Use Ruff (10-100x faster)

```
- name: ⚡ Lint with Ruff
  run: |
    pip install ruff
    ruff check . --fix
```

"Which linter does your team currently use? I can update this right now."

Notebook Integrity Checks

```
      - name: 📓 Validate model_training notebook
        run: |
          python scripts/validate_notebooks.py --notebook ${{ env.NOTEBOOK_PATH }}
          echo "✅ Notebook structure validated"
```

Demo Script

"This is your Notebook Integrity Check requirement. Let me show you what this script validates..."

Validation Checks:

- ✅ Notebook structure (valid JSON, cell formats)
- ✅ Metadata integrity (kernel info, cell IDs)

- ☑ No execution errors in cell outputs
- ☑ Cell count (prevents empty notebooks)
- ☑ Custom business rules you define

🎯 TRAP QUESTION #5

> **Customer:** "What if our notebooks are in Fabric format (.Notebook folder) not Jupyter (.ipynb)?"

YOUR ANSWER: "Perfect observation! Fabric uses a folder structure with `notebook-content.py` inside. We handle both:"

For Fabric notebooks:

```
python scripts/validate_notebooks.py --notebook model_training1.Notebook
```

The script detects the folder structure and validates:

- The `.platform` file exists
- `notebook-content.py` is valid Python
- Dependencies are declared
- Metadata is consistent

For Jupyter notebooks:

```
python scripts/validate_notebooks.py --notebook notebooks/model_training.ipynb
```

Same script, different validators. Want me to show you the Fabric-specific validation code?

Security Scanning (Custom Rules)

```
      - name: 🔐 Check for hardcoded secrets
        run: |
          if grep -r -E '(password|api[_-]?key|secret|token)\s*=\s*["\'][^"\']+
["\']' \
             ${{ env.NOTEBOOK_PATH }} check_data_quality.py --exclude-dir=.git
2>/dev/null; then
            echo "❌ Potential secrets found!"
            exit 1
          fi
          echo "☑ No secrets detected"
```

Demo Script

"This is your Custom Rules requirement - specifically security scanning. We're using regex to catch patterns like `password = "hardcoded123"`. If found, the PR is blocked."

TRAP QUESTION #6

> **Customer:** "Won't this give false positives for variables named 'password' that don't contain secrets?"

YOUR ANSWER: "Absolutely! This is a basic check. For production, I'd recommend:"

Option 1: GitHub Secret Scanning

- Free for public repos
- Included in GitHub Advanced Security
- Scans for 200+ secret patterns
- Uses machine learning to reduce false positives
- Integrates with partner services

Option 2: Advanced Tools

```
- name: 🔐 TruffleHog Secret Scanning
  uses: trufflesecurity/trufflehog@main
  with:
    path: ./
    base: ${{ github.event.pull_request.base.sha }}
    head: ${{ github.event.pull_request.head.sha }}
```

Option 3: detect-secrets

```
- name: 🔐 Detect Secrets
  run: |
    pip install detect-secrets
    detect-secrets scan --baseline .secrets.baseline
```

"Which approach aligns with your security policies? We can upgrade this right now."

Unit Tests

```
      - name: 🧪 Run unit tests
        run: |
          pytest scripts/tests/ -v --tb=short || echo "⚠️ Some tests failed"
          echo "☑️ Unit tests completed"
```

Demo Script

"We run pytest to validate deployment scripts, validation logic, and custom functions."

PR Summary Report

```
      - name: 📊 PR Validation Summary
        if: always()
        run: |
          echo "## ☑ Model Training Notebook - PR Validation" >>
$GITHUB_STEP_SUMMARY
          echo "" >> $GITHUB_STEP_SUMMARY
          echo "| Check | Status |" >> $GITHUB_STEP_SUMMARY
          echo "|-------|--------|" >> $GITHUB_STEP_SUMMARY
          echo "| Code Formatting | ☑ Passed |" >> $GITHUB_STEP_SUMMARY
          echo "| Notebook Validation | ☑ Passed |" >> $GITHUB_STEP_SUMMARY
          echo "| Security Scan | ☑ Passed |" >> $GITHUB_STEP_SUMMARY
          echo "| Unit Tests | ☑ Passed |" >> $GITHUB_STEP_SUMMARY
```

Demo Script

"The summary appears directly in the Pull Request, giving reviewers instant visibility into code quality."

PART 3: DEV Deployment Job (Lines 117-210)

Customer Requirement

> **Step 3 — Deploy to Dev:** When PR is approved & merged, the Deploy-to-Fabric-Dev workflow runs

Implementation

```
  deploy-dev:
    name: 🚀 Deploy to DEV
    if: |
      (github.event_name == 'push' && github.ref == 'refs/heads/main') ||
      (github.event_name == 'workflow_dispatch' && github.event.inputs.environment
== 'dev')
    runs-on: ubuntu-latest
```

Demo Script

"Notice the compound condition:"

- **Automatic:** Runs on every merge to main
- **Manual override:** Developers can manually trigger Dev deployment if needed

🎯 TRAP QUESTION #7

> **Customer:** "What if the DEV deployment fails? Does it block future merges to main?"

YOUR ANSWER: "Great question - this is about deployment vs. merge separation:"

What happens:

1. PR merges to main (code is now in main)
2. DEV deployment starts
3. If deployment fails, code stays in main but DEV workspace isn't updated

Protection strategy:

- The PR validation already caught code issues
- Deployment failures are usually infrastructure (Fabric API timeout, network)
- We have manual retry via `workflow_dispatch`
- Rollback capability (we'll see in PROD deployment)

Advanced option:

- You can add a `required status check` on the DEV deployment if you want to prevent further merges until DEV is healthy

---

Azure Authentication

```
      - name: 🔐 Authenticate to Azure
        uses: azure/login@v2
        with:
          creds: '{"clientId":"${{ secrets.AZURE_CLIENT_ID }}","clientSecret":"${{
secrets.AZURE_CLIENT_SECRET }}","subscriptionId":"${{
secrets.AZURE_SUBSCRIPTION_ID }}","tenantId":"${{ secrets.AZURE_TENANT_ID }}"}'
```

Demo Script

"This is enterprise-grade authentication using Service Principal with Client Secret. All credentials are in GitHub Secrets - zero hardcoded values."

🎯 TRAP QUESTION #8

> **Customer:** "We use Managed Identity in Azure. Can this workflow support that instead of client secrets?"

YOUR ANSWER: "Absolutely! Great security practice. Two options:"

Option 1: Federated Identity (OIDC) - RECOMMENDED

```
- name: 🔐 Authenticate to Azure
  uses: azure/login@v2
  with:
    client-id: ${{ secrets.AZURE_CLIENT_ID }}
    tenant-id: ${{ secrets.AZURE_TENANT_ID }}
    subscription-id: ${{ secrets.AZURE_SUBSCRIPTION_ID }}
```

- No secrets stored!

- GitHub issues short-lived OIDC tokens
- Requires one-time setup in Entra ID

Setup Steps:

1. Create Azure AD App Registration
2. Add Federated Credential → GitHub Actions
3. Configure Subject: `repo:YOUR_ORG/YOUR_REPO:ref:refs/heads/main`

Option 2: Self-hosted runner with Managed Identity

```
runs-on: [self-hosted, azure-vm]
```

- Runner VM has Managed Identity
- Inherits permissions automatically
- No credentials in GitHub

"Which authentication model does your organization prefer? OIDC is becoming the standard - Microsoft recommends it."

Fabric Token Acquisition

```
      - name: 🎟️ Get Fabric Access Token
        run: |
          TOKEN=$(az account get-access-token --resource
https://analysis.windows.net/powerbi/api --query accessToken -o tsv)
          echo "::add-mask::$TOKEN"
          echo "FABRIC_TOKEN=$TOKEN" >> $GITHUB_ENV
```

Demo Script

"This gets a Fabric API token using the Azure authentication we just established. Notice `::add-mask::` - GitHub automatically redacts this token from logs."

🎯 TRAP QUESTION #9

> **Customer:** "Why not use the Fabric REST API directly with the service principal?"

YOUR ANSWER: "You absolutely can! We have two patterns:"

Pattern 1 (Current): Azure CLI → Fabric Token

```
TOKEN=$(az account get-access-token --resource
https://analysis.windows.net/powerbi/api --query accessToken -o tsv)
```

- ☑ Simpler
- ☑ Azure CLI handles token refresh
- ☑ Works with existing Azure auth

Pattern 2: Direct Fabric API

```
from azure.identity import ClientSecretCredential

credential = ClientSecretCredential(
    tenant_id=TENANT_ID,
    client_id=CLIENT_ID,
    client_secret=CLIENT_SECRET
)
token = credential.get_token("https://api.fabric.microsoft.com/.default")
```

- ☑ More control
- ☑ No Azure CLI dependency
- ☑ Better for advanced scenarios

"Both work. Pattern 1 is easier for teams familiar with Azure CLI. Pattern 2 gives more control. Your preference?"

Deployment Execution

```
      - name: 🚀 Deploy model_training notebook to DEV
        id: deploy
        run: |
          echo "🚀 Deploying notebook to DEV workspace..."
          python scripts/deploy_to_fabric.py \
            --workspace-id "${{ secrets.FABRIC_DEV_WORKSPACE_ID }}" \
            --notebook-path "${{ env.NOTEBOOK_PATH }}" \
            --environment dev

          echo "☑ Notebook deployed successfully to DEV"
```

Demo Script

"This is the actual deployment - calling our custom Python script that uses the Fabric REST API."

🏐 TRAP QUESTION #10

> **Customer:** "What exactly does deploy_to_fabric.py do? How does it handle failures?"

YOUR ANSWER: "Let me walk through the deployment logic:"

Script Flow:

```
class FabricDeployer:
    def deploy_notebooks(self, notebooks_path: Path) -> bool:
        # 1. Get Fabric token
        token = self._get_access_token()

        # 2. List existing items in workspace
        existing_items = self._list_workspace_items()

        # 3. Check if item exists
        if notebook_name in existing_items:
            # Update existing
            self._update_notebook(notebook_id, content)
        else:
            # Create new
            self._create_notebook(notebook_name, content)

        # 4. Validate deployment
        self._verify_deployment(notebook_id)

        return True
```

Failure Handling:

1. **Retries with exponential backoff** (network issues)

```
for attempt in range(3):
    try:
        response = requests.post(url, headers=headers, json=payload)
        response.raise_for_status()
        break
    except Exception as e:
        if attempt < 2:
            time.sleep(2 ** attempt)
```

2. **Validates item exists after creation**

```
# Wait for eventual consistency
time.sleep(2)
items = self._list_workspace_items()
assert notebook_id in items
```

3. **Exits with error code**

```
if not success:
    sys.exit(1)  # GitHub Actions marks step as failed
```

4. **Logs detailed error messages**

```
print(f"❌ Deployment failed: {error_message}")
print(f"Response status: {response.status_code}")
print(f"Response body: {response.text}")
```

Idempotency:

- Running deployment twice produces the same result
- Safe to retry on failures
- No duplicate items created

Smoke Tests

```
      - name: ☑ Run smoke tests
        run: |
          echo "🚀 Running smoke tests in DEV..."
          python scripts/run_smoke_tests.py \
            --workspace-id "${{ secrets.FABRIC_DEV_WORKSPACE_ID }}" \
            --environment dev
          echo "☑ Smoke tests passed"
```

Demo Script

"After deployment, we run smoke tests to validate critical functionality:"

What's Tested:

- ☑ Notebook exists and is accessible
- ☑ API health check (Fabric workspace responsive)
- ☑ Data availability (can query sample data)
- ☑ Dependencies loaded correctly

PART 4: PROD Deployment with Governance (Lines 326-475)

Customer Requirement

> **Operating Model & Governance:** Approval gates, Auditability, Role-based access

Implementation

```
  deploy-prod:
    name: 🎯 Deploy to PRODUCTION
    if: github.event_name == 'workflow_dispatch' &&
github.event.inputs.environment == 'prod'
```

```
    runs-on: ubuntu-latest
    environment:
      name: production
```

Demo Script

"Notice `environment: name: production`. This is where GitHub's Environment Protection Rules activate. The workflow literally pauses here and waits for approvals."

TRAP QUESTION #11

> **Customer:** "Who approves Production deployments? Can we require multiple approvers?"

YOUR ANSWER: "Exactly what you want! Here's the governance setup:"

GitHub Settings → Environments → production:

```
☑ Required reviewers: [@senior-engineer, @platform-lead]
  (Requires 2 approvals before deployment proceeds)

☑ Wait timer: 5 minutes
  (Incident response window - can cancel if issues arise)

☑ Deployment branches: main only
  (Cannot deploy from feature branches)

☑ Environment secrets: FABRIC_PROD_WORKSPACE_ID
  (Prod credentials only accessible in prod environment)
```

Audit Trail:

- Every deployment logged in GitHub's Environments page
- Shows WHO approved, WHEN, and WHY (deployment_reason)
- Immutable log for compliance
- Searchable history

This satisfies your "Role-based access" and "Auditability" requirements.

Change Management Integration

```
    - name: 🎫 Validate change ticket
      run: |
        TICKET="${{ github.event.inputs.change_ticket }}"
        if [ -z "$TICKET" ]; then
          echo "❌ Change ticket is required for PROD deployment"
          exit 1
        fi
```

```
            echo "🎫 Change Ticket: $TICKET"
            echo "☑ Change ticket validated"
```

Demo Script

"This enforces your Change Management process. Production deployments MUST have a ticket number (e.g., CHG-12345 from ServiceNow or Jira)."

TRAP QUESTION #12

> **Customer:** "Can we validate the ticket actually exists in ServiceNow before deploying?"

YOUR ANSWER: "Absolutely! That's governance best practice. Here's how:"

```
- name: 🎫 Validate change ticket in ServiceNow
  run: |
    TICKET="${{ github.event.inputs.change_ticket }}"

    # Query ServiceNow API
    RESPONSE=$(curl -s -u "${{ secrets.SERVICENOW_USER }}:${{
secrets.SERVICENOW_PASS }}" \
      "https://yourinstance.service-now.com/api/now/table/change_request?
number=$TICKET&sysparm_fields=state,approval")

    STATE=$(echo $RESPONSE | jq -r '.result[0].state')
    APPROVAL=$(echo $RESPONSE | jq -r '.result[0].approval')

    if [ "$STATE" != "Implement" ] || [ "$APPROVAL" != "approved" ]; then
      echo "❌ Change ticket $TICKET is not approved or not in Implement state"
      exit 1
    fi

    echo "☑ Change ticket $TICKET is valid and approved"
```

Additional Integrations:

- ☑ Update ticket status after deployment ("Implementation Complete")
- ☑ Attach deployment logs to the ticket
- ☑ Validate ticket is scheduled for current time window
- ☑ Create related incident if deployment fails

Supported ITSM Platforms:

- ServiceNow
- Jira Service Management
- BMC Remedy
- Cherwell

"Do you use ServiceNow, Jira, or another ITSM platform?"

Production Backup

```
      - name: 📸 Backup current PROD state
        id: backup
        run: |
          echo "📸 Creating backup of PROD workspace..."
          BACKUP_ID="backup-$(date +%Y%m%d-%H%M%S)"
          echo "backup_id=$BACKUP_ID" >> $GITHUB_OUTPUT

          python scripts/backup_workspace.py \
            --workspace-id "${{ secrets.FABRIC_PROD_WORKSPACE_ID }}" \
            --backup-id "$BACKUP_ID" \
            --output-path "backups/"

          echo "☑ Backup created: $BACKUP_ID"
```

Demo Script

"Before touching Production, we create a backup. This satisfies your 'Operating Model & Governance' requirement for safety."

🎯 TRAP QUESTION #13

> **Customer:** "Where is the backup stored? How long do you keep it? Can we restore from it?"

YOUR ANSWER: "Critical questions for DR planning. Here's the strategy:"

Storage Locations (configurable):

```
# Local (GitHub Actions artifacts)
--output-path "backups/"

# Azure Blob Storage
--output-path "az://fabricbackups/prod"

# AWS S3
--output-path "s3://fabric-backups/prod"

# Google Cloud Storage
--output-path "gs://fabric-backups/prod"
```

What's Backed Up:

- ☑ Notebook definitions (code + metadata)
- ☑ Pipeline JSON configurations
- ☑ Lakehouse schemas
- ☑ Workspace metadata (items, permissions)
- ☑ Semantic models (if applicable)

Retention Policy:

```
- name: 📦 Upload backup to long-term storage
  uses: actions/upload-artifact@v3
  with:
    name: prod-backup-${{ steps.backup.outputs.backup_id }}
    path: backups/
    retention-days: 90  # Configurable: 1-400 days
```

Enterprise Storage (Azure Blob):

```
- name: 📦 Upload to Azure Blob with immutability
  run: |
    az storage blob upload \
      --account-name fabricbackups \
      --container-name prod \
      --name $BACKUP_ID.zip \
      --file backups/$BACKUP_ID.zip \
      --tier Cool \
      --immutability-policy \
        --immutability-period-since-creation-in-days 2555  # 7 years
```

Restoration Process:

```
# Manual restore
gh workflow run model-training-pipeline.yml \
  -f environment=prod \
  -f restore_backup=backup-20260106-143022 \
  -f deployment_reason="Rollback due to incident INC-67890" \
  -f change_ticket="CHG-EMERGENCY-001"
```

"For enterprise compliance, I'd recommend Azure Blob with immutable storage. Want me to show that setup?"

PART 5: Automatic Rollback & DR (Lines 484-519)

Customer Requirement

> **Operating Model:** Disaster recovery, incident response

Implementation

```
  rollback:
    name: ⬅️ Rollback Deployment
    if: failure() && needs.deploy-prod.result == 'failure'
```

```
    needs: [deploy-prod]
    runs-on: ubuntu-latest
```

Demo Script

"Notice three critical things:"

1. `if: failure()` - Only runs if something went wrong
2. `needs.deploy-prod.result == 'failure'` - Specifically watching PROD deployment
3. `needs: [deploy-prod]` - Creates dependency chain

TRAP QUESTION #14

> **Customer:** "What if the rollback itself fails? Do we have a rollback for the rollback?"

YOUR ANSWER: "Excellent question - this is the 'who watches the watchmen' problem. Enterprise strategy:"

Layer 1: Automated Rollback (Workflow)

```
- name: 🔙 Execute rollback
  run: |
    python scripts/rollback_deployment.py \
      --workspace-id "${{ secrets.FABRIC_PROD_WORKSPACE_ID }}" \
      --backup-id "${{ needs.deploy-prod.outputs.backup_id }}"
```

- ☑ Runs automatically on deployment failure
- ☑ Uses the backup created moments ago
- ☑ Fast recovery (< 5 minutes)

Layer 2: Rollback with Emergency Bypass

```
- name: 🔙 Execute rollback
  run: |
    python scripts/rollback_deployment.py \
      --workspace-id "${{ secrets.FABRIC_PROD_WORKSPACE_ID }}" \
      --backup-id "${{ needs.deploy-prod.outputs.backup_id }}" \
      --skip-validation  # ← Emergency bypass
      --force           # ← Skip confirmations
  continue-on-error: true  # ← Don't fail if rollback fails
```

Layer 3: Manual Intervention

```
- name: 🚨 Alert on-call team
  if: failure()
  uses: 8398a7/action-slack@v3
  with:
```

```
      status: failure
      text: |
        🚨 PROD deployment AND rollback failed!
        Backup ID: ${{ needs.deploy-prod.outputs.backup_id }}
        Manual intervention required: https://runbook.example.com/fabric-dr
```

Layer 4: Disaster Recovery

- PagerDuty/Teams alert to on-call engineer
- Manual restoration from Azure Blob backup
- Fabric workspace-level restore (if available)
- Documented runbook for worst-case scenario

Runbook Example:

```
## DR Procedure: Manual Rollback

1. Download backup from Azure:
   `az storage blob download --name backup-20260106-143022.zip`

2. Extract backup:
   `unzip backup-20260106-143022.zip`

3. Restore to Fabric:
   `python scripts/restore_backup.py --backup-path ./backup-20260106-143022`

4. Verify restoration:
   `python scripts/validate_deployment.py --workspace-id PROD_WORKSPACE_ID`

5. Update incident ticket:
   ServiceNow: INC-67890
```

"Most organizations have ~3 layers. What's your current incident response process? We can align this workflow to it."

PART 6: Demo Summary & Alignment

Requirements Checklist

Requirement	Implementation	Location
☑ **Step 1: Local Development**	Developer works locally, commits to feature branch	Standard Git workflow
☑ **Step 2: Pull Request**	Auto-triggers on PR	Lines 4-11
☑ **Linting**	Black formatter	Lines 69-72
☑ **JSON Schema Checks**	Pipeline validation	validate_pipelines.py

Requirement	Implementation	Location
☑ **Notebook Integrity**	Custom validator	Lines 74-77
☑ **Custom Rules**	Security scan	Lines 79-86
☑ **Unit Tests**	Pytest	Lines 88-92
☑ **PR Summary**	GitHub Actions summary	Lines 94-109
☑ **Step 3: Deploy to Dev**	Auto-deploy on merge	Lines 14-18, 117-210
☑ **Smoke Tests**	Post-deployment validation	Lines 179-184
☑ **Step 4: Promote to TEST**	Manual trigger	Lines 212-320
☑ **Step 4: Promote to PROD**	Manual trigger with approvals	Lines 326-475
☑ **Approval Gates**	Environment protection rules	Line 330
☑ **Change Management**	Ticket validation	Lines 337-343
☑ **Production Backup**	Pre-deployment backup	Lines 368-380
☑ **Automatic Rollback**	Failure detection & restore	Lines 484-519
☑ **Auditability**	GitHub Actions logs + Environment history	Built-in
☑ **Role-based Access**	Environment reviewers	GitHub Settings

TRAP QUESTIONS REFERENCE GUIDE

Security & Authentication

Q1: "What about secret rotation?" **A:** GitHub Secrets can be rotated without code changes. Use Azure Key Vault integration for automatic rotation.

Q2: "How do we audit who accessed secrets?" **A:** GitHub audit log tracks all secret access. Enterprise tier provides SIEM integration.

Q3: "What about secrets in logs?" **A:** GitHub automatically masks registered secrets. We use `::add-mask::` for dynamic values.

Scale & Performance

Q4: "How many deployments per day can this handle?" **A:** GitHub Actions limits:

- Free tier: 2,000 minutes/month
- Team: 3,000 minutes/month
- Enterprise: 50,000 minutes/month
- Self-hosted: Unlimited

Q5: "What if Fabric API has rate limits?" **A:** Our deployment script implements exponential backoff and respects Fabric rate limits (currently 200 requests/minute per tenant).

Q6: "Can we deploy to multiple workspaces in parallel?" **A:** Yes! Use matrix strategy:

```
strategy:
  matrix:
    workspace:
      - { id: 'abc-123', name: 'Prod-US' }
      - { id: 'def-456', name: 'Prod-EU' }
```

Compliance & Governance

Q7: "Does this satisfy SOC 2 requirements?" **A:** Yes, with proper configuration:

- Audit trails (GitHub Actions logs)
- Access controls (Environment protection)
- Change management (Ticket validation)
- DR/BCP (Backup + rollback)

Q8: "What about GDPR data residency?" **A:** Use GitHub Enterprise Server (on-premises) or self-hosted runners in your region.

Q9: "How long are deployment logs retained?" **A:**

- Workflow logs: 90 days (configurable up to 400)
- Artifacts: 90 days default
- For compliance, export to Azure Monitor/Splunk

Cost Optimization

Q10: "What's the monthly cost?" **A:** Example calculation:

- GitHub Actions (Team): $4/user/month
- Azure Service Principal: Free
- Fabric API calls: Free
- Storage (artifacts): ~$0.10/GB/month
- **Total for 10-person team: ~$50/month**

Q11: "Can we reduce Actions minutes usage?" **A:**

- Use self-hosted runners (free)
- Optimize workflows (parallel jobs)
- Use caching aggressively
- Skip unnecessary steps with conditions

Enterprise Adoption

Q12: "How do we roll this out to 50 teams?" **A:** Phased approach:

1. **Week 1-2:** Pilot with 1 team (1 notebook)
2. **Week 3-4:** Expand to 3 teams (5 notebooks each)
3. **Month 2:** Create reusable workflow template
4. **Month 3:** Self-service onboarding for remaining teams

Q13: "What training is required?" **A:**

- Developers: 2-hour workshop (Git basics + workflow triggers)
- DevOps: 1-day deep dive (Actions syntax + Fabric API)
- Approvers: 30-min session (approval process)

Next Steps Framework

Decision Point 1: POC vs. Pilot

POC (2 weeks):

- ☑ 1 notebook
- ☑ DEV environment only
- ☑ Basic validation (linting + notebook check)
- ☑ Manual deployment
- **Success Criteria:** Deployment works end-to-end

Pilot (1 month):

- ☑ 5 notebooks
- ☑ DEV → TEST → PROD
- ☑ Full validation suite
- ☑ Approval gates enabled
- ☑ 1 production deployment completed
- **Success Criteria:** Team adopts workflow, 90% automation rate

Full Rollout (3 months):

- ☑ All notebooks/pipelines
- ☑ Multiple workspaces
- ☑ ITSM integration (ServiceNow)
- ☑ Custom validators per team
- ☑ Self-service onboarding
- **Success Criteria:** 100% of deployments via pipeline, < 5% rollback rate

---

Decision Point 2: Authentication Strategy

Option	Pros	Cons	Best For
Client Secret	Simple, works everywhere	Secrets to rotate	Quick start

Option	Pros	Cons	Best For
OIDC (Federated)	No secrets, short-lived tokens	Requires Entra setup	Production
Managed Identity	Zero secrets, Azure-native	Needs self-hosted runners	Enterprise

Recommendation: Start with Client Secret for POC, migrate to OIDC for production.

Decision Point 3: Approval Model

Model	Configuration	Use Case
Basic	1 reviewer, no wait time	Small teams, trusted devs
Standard	2 reviewers, 5-min wait	Most organizations
Enterprise	3+ reviewers, 30-min wait, business hours only	Financial/Healthcare

---

Closing Questions

For the Customer:

1. **Alignment Check:** "Does this CI/CD model meet your governance and automation expectations?"
2. **Constraints:** "Any concerns about security, scale, or environment setup?"
3. **Next Steps:** "Would you prefer to:"
    - Start with a 2-week POC (1 notebook, prove the concept)
    - Jump to a 1-month Pilot (5 notebooks, full workflow)
    - Custom approach based on your constraints
4. **Timeline:** "What's your target date for having this in production?"
5. **Success Metrics:** "How will you measure success of this CI/CD implementation?"
    - Deployment frequency?
    - Mean time to recovery?
    - Developer satisfaction?
    - Compliance audit pass rate?

Appendix: Quick Command Reference

Trigger Deployments

```
# Deploy to DEV (manual)
gh workflow run model-training-pipeline.yml \
  -f environment=dev \
  -f deployment_reason="Testing new feature"
```

```
# Deploy to TEST
gh workflow run model-training-pipeline.yml \
  -f environment=test \
  -f deployment_reason="Promote from DEV"

# Deploy to PROD
gh workflow run model-training-pipeline.yml \
  -f environment=prod \
  -f deployment_reason="Production Release v2.1" \
  -f change_ticket="CHG-12345"
```

Monitor Deployments

```
# List recent runs
gh run list --workflow=model-training-pipeline.yml --limit 5

# Watch live run
gh run watch

# View specific run
gh run view 20735157003

# Download logs
gh run download 20735157003
```

Manage Environments

```
# List environment deployments
gh api repos/:owner/:repo/environments/production/deployments

# View environment protection rules
gh api repos/:owner/:repo/environments/production
```

Document Version: 1.0
Last Updated: January 6, 2026
Prepared For: Customer Demo - Fabric CI/CD with GitHub Actions
Contact: Your Implementation Team